A Verifiable RLVR Gym for Sequential Planning Under Uncertainty

Imen — *Internal Note, April 8, 2026*

1. The Gap in Current RLVR

Every RLVR gym today tests **static reasoning**: the model sees a complete problem, produces one answer, gets scored. Math, code, logic puzzles — all follow this pattern. No existing gym tests **sequential decision-making under uncertainty**, where the agent must act before the full picture is revealed, each action has a cost, and the optimal strategy depends on future dynamics.

This is a distinct cognitive capability. Consider:

- **Static (math gym):** "The expected return is 0.105 and the fee is 0.15. Should you enter?" → One computation: no.
- **Sequential (our gym):** Same numbers, but the signal persists. Enter now, pay 0.15 once, earn returns for many steps. The answer depends on *how long the signal will persist* — requiring reasoning about dynamics, not just the current snapshot.

The sequential version requires amortisation reasoning: weighing a one-time cost against uncertain future gains. This is the planning capability we target.

2. Why Finance — and Why It's Been Hard

Finance is a natural domain for sequential planning: a portfolio manager deciding whether to enter a trade weighs immediate profit against entry cost, persistence of the opportunity, and exit cost if conditions change. But finance has been resistant to RLVR for a fundamental reason: **you cannot compute the optimal policy when you don't know the true distribution.** Real markets have unknown dynamics, so there is no ground-truth reward to verify against. Existing finance RL environments (e.g., FinRL) use simulation-based evaluation — fine for standard RL, unusable for RLVR.

This is not a limitation of our approach; it is a requirement of RLVR itself. Math gyms use generated equations with known answers. Reasoning Gym uses procedural puzzles. We use a parametric financial model with an exact analytical solution.

3. Our Setup

We build on Bilokon (2026), who formalises a simple but powerful idea: at each time step, an agent observes a noisy signal and decides whether to **activate** (ON) or **deactivate** (OFF) a strategy. Being ON collects the strategy's profit-and-loss (PnL); being OFF earns nothing. Each switch (ON→OFF or OFF→ON) costs a fixed transaction fee λ.

The signal follows an Ornstein-Uhlenbeck (OU) process — it fluctuates around zero with a tendency to revert back. When the signal is positive, being ON is profitable. When it's negative, being ON loses money. The key parameter is κ (mean-reversion speed): low κ means the signal persists (today's value predicts tomorrow's); high κ means it reverts quickly (nearly unpredictable one step ahead).

Why OU? The OU process has Gaussian transitions, which means we can solve the Bellman equation *exactly* via Gauss-Hermite quadrature. No approximation, no Monte Carlo noise — a provably optimal policy for every parameter configuration. This is what gives us the verifiable reward.

Why binary ON/OFF? Bilokon (2026) proves that a single bit — ON or OFF — is the information-theoretically optimal compression of the full market state for predicting strategy performance. It's not a simplification; it's the theoretically justified action space.

4. Where Planning Matters

Without switching costs, the problem is trivial: ON when signal > 0, OFF otherwise. With switching costs, the agent must reason about persistence: "The signal is slightly below my break-even threshold right now, but it's been positive for several steps. Should I pay the cost to switch ON, expecting to earn over multiple future steps?"

This is exactly the planning we want to train.

We ran a full parameter sweep across 1,500 configurations and found:

- **κ is the dominant factor.** It alone explains 46% of the variance in how often the optimal policy disagrees with greedy. At $\kappa = 0.1$ (persistent signal), approximately 13% of decisions require planning. At $\kappa = 0.7$ (fast reversion), only ~1.2% do.
- **The λ/α ratio has a non-monotone effect.** At very low switching costs, both policies agree (switch freely). At very high costs, both agree (never switch). The planning gap peaks at intermediate costs ($\lambda/\alpha \in [0.3, 0.5]$).
- **This gives us parametric control over difficulty.** We define a "planning zone" ($\kappa \in [0.1, 0.25]$, $\lambda/\alpha \in [0.3, 0.5]$) where episodes produce meaningful gradient signal, and a "fast-reversion zone" ($\kappa = 0.7$) as a control condition.

This is analogous to a math gym excluding single-digit addition from the training distribution: episodes where greedy equals optimal produce no training signal for planning.

5. Evaluation Methodology

We introduce a **two-bucket methodology** that separates two capabilities:

- **Easy decisions** (greedy = optimal): Tests comprehension — can the model correctly evaluate the immediate cost-benefit? These are approximately 95% of all decisions.
- **Hard decisions** (greedy $\neq$ optimal): Tests planning — can the model override the locally attractive greedy action when forward-looking reasoning says otherwise?

We further verify hard decisions via **CoT inspection**: does the model's chain-of-thought actually contain forward-looking reasoning (trend recognition, horizon counting, amortisation), or did it match optimal by accident?

6. Results: Multi-Model Evaluation (N=500)

We scaled to N=500 episodes per model (same seeds, same planning zone) to get statistically significant results. Three Anthropic models evaluated; GPT-4o ongoing.

Comprehension (easy decisions) All models: **98.4–98.5%**. Indistinguishable. Comprehension is solved.

Raw hard match rate

Model	Hard decisions	Raw match rate	95% CI
Opus 4	551	9.1%	[7.0%, 11.8%]
Haiku 4.5	548	7.5%	[5.6%, 10.0%]
Sonnet 4	559	3.9%	[2.6%, 5.9%]

Opus vs Sonnet: p=0.001. But Opus vs Haiku: p=0.338. The raw rates are contaminated by accidental matches.

CoT-verified genuine planning (the real metric) We classify a hard match as "genuine planning" only if the CoT contains forward-looking framing AND amortisation logic (multi-step cost recovery). This filters out tiny-margin luck and borderline matches.

Model	Genuine planning rate	95% CI
Opus 4	**3.1%** (17/551)	[1.9%, 4.9%]
Haiku 4.5	**0.5%** (3/548)	[0.2%, 1.6%]
Sonnet 4	**0.4%** (2/559)	[0.1%, 1.3%]

Opus vs Haiku: p=0.0016. Opus vs Sonnet: p=0.0005. CIs don't overlap.

The gym cleanly discriminates Opus from both mid-tier models. Opus computes multi-step amortisation ("0.1 per step $\times$ 3 steps = 0.3 > switching cost"). Haiku and Sonnet never do — their matches are accidental.

Domain transfer Energy dispatch prompt (same math, different words): comparable results. No domain-specific hallucinations. The bottleneck is sequential reasoning, not financial knowledge.

The J-based reward The regret-normalised J score does **not** discriminate models at evaluation scale (all $p > 0.2$). Hard decisions are too rare (~1 per episode) for their economic impact to separate models on J. Important distinction:

- **For evaluation:** use CoT-verified planning rate (discriminates at N=500)
- **For training:** use J-based reward (correct gradient signal, accumulates over thousands of episodes)

7. Why This Matters for RLVR Training

Opus's 3.1% genuine planning rate is low but non-zero — the capability exists in the model's distribution.

GRPO viability: With batch size K=32, approximately 67% of episodes containing hard decisions will have at least one completion that outperforms greedy. The rate is expected to increase during training.

Cold-start SFT: For models with near-zero planning rates (Haiku 0.5%, Sonnet 0.4%), direct GRPO may not produce sufficient positive signal. The gym provides a natural SFT pipeline: 1. Run Opus on ~10,000 episodes → ~310 genuine planning examples (at 3.1%) 2. SFT the target model on these → bootstraps the planning rate 3. GRPO takes over → pushes it higher

This is analogous to DeepSeek-R1's approach: SFT on long CoT examples, then GRPO.

Integration is minimal: one function `reward_fn(completions, problems) → list[float]`, compatible with TRL/veRL GRPO pipelines.

8. Beyond Finance

The planning capability tested here — amortising a one-time cost against uncertain future gains under a noisy, persistent signal — is not specific to finance. The same structure appears in:

- **Energy dispatch:** activating/deactivating generators with startup costs under fluctuating demand
- **Supply chain:** onboarding/offboarding suppliers with switching costs under variable demand signals
- **Treatment protocols:** starting/stopping treatments with side-effect costs under evolving patient indicators

The domain-specific prompt result supports this: the bottleneck is sequential reasoning, not domain knowledge. Training on the financial gym could develop planning capabilities that transfer to any domain with the same cost-amortisation structure. The framework extends to other signal processes and PnL functions by changing only the transition kernel and payoff mapping.

9. Current Status

- ✓ Gym implemented, tested, open-source
- ✓ Bellman solver verified (T=1 reduces to greedy, threshold matches analytical prediction)
- ✓ 1,500-config parameter sweep completed
- ✓ Multi-model evaluation (N=500): Opus, Haiku, Sonnet — statistically significant discrimination
- ✓ Domain transfer test (energy dispatch): confirms domain-agnostic
- ✓ CoT classification pipeline (automated, sensitivity-tested)
- ✓ Single-function TRL integration
- ✓ SFT data pipeline: 17 genuine planning examples from Opus, scalable to ~300 with 10K episodes
- ☐ RLVR training not yet conducted
- ☐ GPT-4o evaluation ongoing

The gym is ready for training experiments. Opus plans genuinely on 3.1% of hard decisions — sufficient to seed GRPO with K>=32. Cold-start SFT is available for models starting near zero. The question is whether RLVR can push the 3.1% higher — and whether the capability transfers.